

Recursividad: apuntes

1. Definición.
 2. Diseño e implementación de algoritmos recursivos.
 3. Mecánica de la recursividad.
 4. Uso de la pila en el diseño de módulos recursivos.
 5. Quicksort
-

1. Definición

Cualquier proceso puede ser definido de forma iterativa y de forma recursiva.

- ⇒ Un proceso iterativo es aquél que requiere de la repetición explícita de cierta acción.
- ⇒ Un proceso es recursivo si está definido total o parcialmente en términos de sí mismo.

Para calcular $4!$, por ejemplo, podemos utilizar un proceso iterativo o uno recursivo.

De manera iterativa:

$$4! = 1 * 2 * 3 * 4 = 24$$

De talo forma que tenemos que hacer 3 multiplicaciones, es decir, la acción que repetimos es la multiplicación.

Para definir $4!$ de manera recursiva tenemos que definirlo en términos de factorial, es decir, en la definición (parte derecha de la igualdad) tiene que aparecer el factorial.

De manera recursiva:

$$4! = 4 * 3!$$

Note que $4!$ lo definimos en términos de $3!$ En la definición iterativa, al hacer las multiplicaciones, obtenemos que $4!$ es igual a 24, mientras que, hasta ahora, con la definición recursiva anterior todavía no podemos dar este resultado. Esto es porque ahora tendríamos que decir qué es $3!$ lo cual nos va a llevar a decir qué es $2!$ y así sucesivamente. A continuación se muestra este proceso y cómo obtenemos el resultado deseado.

$4! = 4 * 3! = 4 * 6 = 24$	Sustituyendo $3!$
$3! = 3 * 2! = 3 * 2 = 6$	Sustituyendo $2!$
$2! = 2 * 1! = 2 * 1 = 2$	Sustituyendo $1!$
$1! = 1 * 0! = 1 * 1 = 1$	Sustituyendo $0!$
$0! = 1$	

Observemos que en el cálculo de $4!$ fue muy importante el hecho de que $0!$ es igual a 1 ¿Qué hubiera pasado si $0!$ se calculara como calculamos los demás factoriales? Nunca habiésemos terminado el proceso, o sea, tendríamos un proceso infinito.

```
if (factorial(4) < valor)
```

Primeramente se debe calcular el factorial de 4 y posteriormente comparar el resultado con la variable "valor". De tal forma que la instrucción que se debe realizar después de la llamada es el operador de comparación <.

Supongamos que queremos imprimir los primeros n naturales en forma descendente. Primeramente tenemos que encontrar la definición recursiva del problema.

1. ¿Cuál es el natural más pequeño que debemos imprimir?

El natural más pequeño es el 1, ya que los números naturales son los enteros positivos.

2. ¿Cómo se conoce el natural $(n - 1)$ suponiendo que se conoce el natural n ?

En este caso consideramos que conocemos n y que el siguiente natural que deseamos es el $(n - 1)$ porque los queremos imprimir en forma descendente. Sabemos que el natural anterior a uno dado es ese natural menos uno, por lo que $(n - 1) = n - 1$.

De tal forma que la definición recursiva de la impresión de los primeros n naturales es:

- a) $\text{imprimeNaturales}(n) \rightarrow \text{imprimir "1"}$ si $n = 1$ $\Rightarrow$ Definición base
- b) $\text{imprimeNaturales}(n) \rightarrow \text{imprimir } n$ si $n > 1$ $\Rightarrow$ Definición recursiva
- $\text{imprimeNaturales}(n - 1)$

Note que en esta definición recursiva no utilizamos la asignación, sino que en su lugar usamos “ $\rightarrow$ ”. Esto es porque `imprimeNaturales` realiza un proceso pero no calcula ningún valor, es decir, es un procedimiento y no una función, por lo que no tendrá *return*.

La implementación de la definición anterior es:

```

procedure imprimeNaturales(n:integer);
begin
    if (n = 1)
        writeln("1");
    else
        -begin
            writeln(n);
            imprimeNaturales(n - 1);
        -end;
end;

```

En el *main* la llamada al procedimiento sería, por ejemplo:

```
imprimeNaturales(5);
writeln("Fin del proceso");
```

Después de terminada la ejecución de `imprimeNaturales(5)` la instrucción que se debe realizar es la impresión de "Fin del proceso".

Conclusiones:

- ① La solución recursiva de un problema se obtiene respondiendo dos preguntas: ¿cómo se resuelve el caso más pequeño? y ¿cómo se resuelve un caso general, sabiendo que ya se tiene el caso anterior más pequeño?
- ② Para implementar soluciones recursivas se utilizan los módulos: procedimientos () y las funciones (integer, real, etc.).
- ③ El *return* de una función realiza dos operaciones: asigna a la función el valor que regresa y termina la ejecución de la misma, regresando el control a la siguiente instrucción que se tiene que realizar, después de la llamada a la función.
- ④ Los procedimientos no tienen *return*, por lo que no regresan ningún valor.

3. Mecánica de la recursividad.

La mecánica de la recursividad está basada en una “pila”. Cuando un módulo recursivo se está ejecutando se crea en la memoria de la computadora una pila donde se almacenan los valores de los parámetros y de las variables locales del módulo. Si el módulo es función también se guarda en la pila el valor que adquiere la misma.

Para cada llamada del módulo se almacenan en la pila los nuevos valores de los parámetros y de las variables locales, creándose un nuevo “registro de activación”. De tal forma que, la pila de recursión está formada por registros de activación. Al terminar una llamada al módulo, es decir, cuando se cumple la definición base, se libera (sale) el registro de activación que se encuentra en el tope de la pila. De esta forma es como puede “recordar” qué valores tenían los parámetros y las variables locales en la llamada anterior.

Si observamos el proceso que seguimos para calcular $4!$ vemos que el valor de n fue cambiando conforme fuimos entrando en recursión y que al salir de recursión necesitábamos recordar el valor que tenía n en la expresión anterior. Esto quiere decir que los valores que fue adquiriendo n fueron entrando a la pila.

No sólo debe recordar los valores que tenían los parámetros y las variables locales al realizarse la correspondiente llamada al módulo sino que también tiene que recordar qué instrucción debe realizar al terminar esa llamada. De tal forma que los registros de activación están compuestos básicamente de:

1. Instrucción a la que debe regresar el control una vez terminada la ejecución actual del módulo.
2. Todos los parámetros y variables locales del módulo.
3. Si el módulo recursivo es una función el valor que adquiere la misma, ya que éste se debe regresar.

Para hacer la representación de la pila de recursión numeramos las instrucciones a las que debe regresar el control una vez terminada la ejecución del módulo recursivo y estos valores son los que ponemos en la pila.

Para ejemplificar la representación de la pila vamos a utilizar la función “factorial”.

```
function factorial (n:integer):integer
begin
    if (n = 0)
        factorial:= 1;
    else
        factorial:= n * factorial(n - 1); _____ 2
    end;
Program Prueba;
begin
    ....
    resultado = factorial(4); _____ 1
    writeln("Resultado = " + resultado);
    ....
end;
```

Instrucción de regreso	n	factorial	
1	4	4 * 6 = 24	
2	3	3 * 2 = 6	
2	2	2 * 1 = 2	
2	1	1 * 1 = 1	
2	0	1	

Ahora veamos cómo es la pila de los registros de activación de un procedimiento, para lo cual utilizaremos “imprimeNaturales”:

```
procedure imprimeNaturales(int n)
begin
    if (n = 1)
        writeln("1");
    else
        writeln(n);
        imprimeNaturales(n - 1);
        _____ 2
    end;
Program Prueba;
begin
    ....
    imprimeNaturales(5);
    writeln("Fin del proceso"); _____ 1
end;
```

Instrucción de regreso	n	
1	5	
2	4	
2	3	
2	2	
2	1	

Observe en este ejemplo que la instrucción que numeramos como la 2 es la llave que cierra el “else”, esto significa que una vez que se termina la ejecución de la llamada al procedimiento imprimeNaturales($n - 1$) la instrucción que se tiene que ejecutar es terminar el “else”.

Consideremos un ejemplo más, la serie de los números de Fibonacci cuya definición ya se nos da de manera recursiva. Vamos a escribir su implementación y posteriormente haremos la representación de la pila.

$$\begin{aligned} F_0 &= 1 \\ F_1 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \quad \text{si } n > 1 \end{aligned}$$

```
function fibonacci(n:integer):integer;
begin
  if (n < 2)
    fibonacci:= 1;
  else
    fibonacci:= fibonacci(n - 1) + fibonacci(n - 2);
  end;
end;
Program Prueba;
begin
  writeln(Fibonacci(4));
end
```

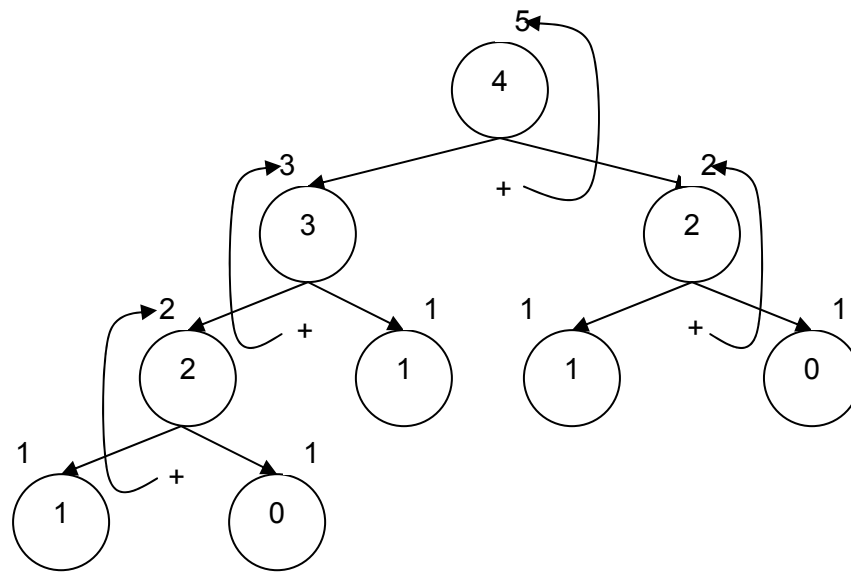
Instrucción de regreso	n	fibonacci	
1	4	3 + 2 = 5	← Resultado
2a	3	2 + 1 = 3	
2a	2	1 + 1 = 2	
2a	1	1	
2b	0	1	
2b	1	1	
2b	2	1 + 1 = 2	
2a	1	1	
2b	0	1	

En la implementación de Fibonacci vemos que en el segundo *return* (definición recursiva) llamamos dos veces a la función (2a y 2b). Cuando un módulo se llama más de una vez a si mismo se dice que es recursivo múltiple.

El orden de las operaciones que se tienen que realizar antes de poderse ejecutar el *return* es:

- 1^o. Calcular fibonacci($n - 1$). El valor obtenido se queda en espera.
- 2^o. Calcular fibonacci($n - 2$). El valor obtenido “se va” al registro donde está en espera el obtenido en el primer punto.
- 3^o. Se suman los valores obtenidos en el primer y en el segundo.

La ejecución de Fibonacci(4) gráficamente, no formalmente, la podemos ver de la siguiente manera:



Los registros de activación están ocupando espacio en la memoria por lo que es muy importante que al diseñar un módulo recursivo utilicemos la menor cantidad de parámetros y de variables locales posibles.

Realizar manualmente la pila de los registros de recursión nos puede ayudar no tan sólo para saber si el módulo es correcto sino que también para diseñar un módulo recursivo.

Conclusiones:

- ① La mecánica de la recursividad es una pila, es decir, al ejecutarse un módulo se crea en la memoria una pila.
- ② Los elementos que componen la pila se llaman “registros de activación” los cuales están compuestos de: instrucción a la que debe regresar el control, los parámetros y las variables locales. Si el módulo es función también se guardan en los registros los valores que va adquiriendo.
- ③ En cada llamada del módulo se crea un nuevo registro y al terminar una ejecución del módulo se libera un registro.
- ④ Los módulos recursivos deben tener la menor cantidad posible de parámetros y variables locales para no consumir demasiado espacio de memoria.

4. Uso de la pila en el diseño de módulos recursivos.

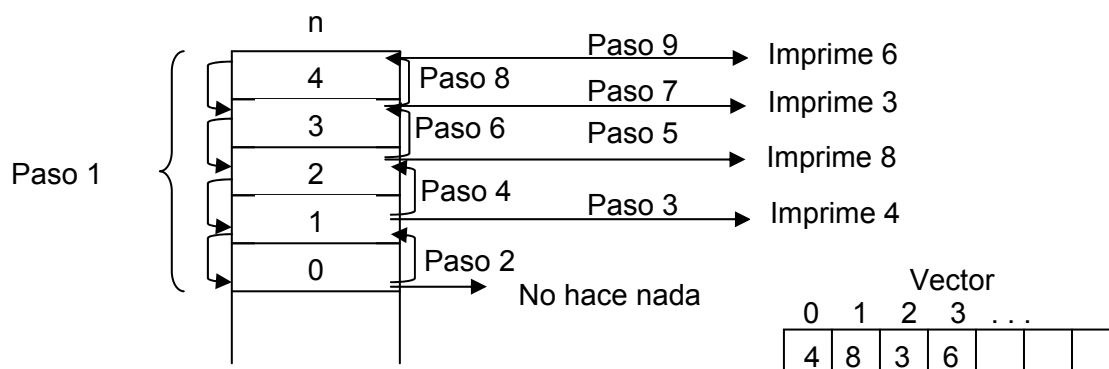
Normalmente las operaciones que realizamos en un vector las hacemos de manera iterativa, pero también podemos resolverlas usando recursividad.

Suponga que tenemos un vector que contiene n enteros y deseamos imprimir los n elementos de manera recursiva, imprimiendo del último al primero.

Cuando tenemos un vector, no sólo conocemos el vector sino que también sabemos que tiene n elementos y que el primer índice del mismo es 0. De tal forma que al llamar al módulo le enviaremos n (cantidad de elementos almacenados), por lo que n será el primer valor que entre a la pila. Si $n > 0$, hay elementos en el vector, nos interesa imprimir el último elemento del vector, con lo cual podemos pensar que ahora el vector tiene un elemento menos, es decir, ahora n en realidad es $n - 1$ y repetimos el proceso. Esta repetición del proceso la va a realizar la llamada recursiva. En este punto debemos preguntarnos ¿cuándo termina de repetirse el proceso? Cuando n sea 0 ya no queremos hacer nada, es decir, la definición base será implícita.

```
procedure imprimeUltimoPrimero(vec:vector, n:integer);
begin
    if (n > 0)
        writeln(vec[n - 1]); // Imprime el último del vector.
        imprimeUltimoPrimero(vec, n - 1); // Repite el proceso
                                   // desconsiderando el que ya
                                   // se imprimió.
    end;
                                   // Si n = 0 no hace nada (no
                                   // hay else)
```

Si ahora queremos imprimir los elementos del vector pero del primero al último ¿qué podemos hacer? Sabemos que a la pila entra n cuando llamamos por primera vez al módulo, pero en ese momento no queremos imprimir nada, ya que necesitamos que n sea 1 para poder imprimir el elemento que está en el índice 0, siempre y cuando haya elementos en el vector, después imprimir el que está en el índice 1 y así sucesivamente hasta imprimir el que está en $n - 1$. Esto nos sugiere que podemos llamar al módulo repetidamente decrementando el valor de n hasta llegar a 1 y entonces imprimir los elementos del vector conforme n regresa al valor que tenía originalmente. Supongamos que tenemos un vector con 4 elementos, entonces gráficamente, utilizando una pila, podemos ver este proceso de la siguiente manera:



El paso 1 indica que queremos que entren en la pila los valores de n , comenzando en 4 y decrementándola en 1 hasta llegar a 0. Cada vez que entra un nuevo valor en la pila se crea un nuevo registro de activación, ¿cómo creamos un nuevo registro? Llamando al módulo. Esto quiere decir que antes de imprimir el elemento del vector tenemos que llamar al módulo.

Cuando n alcanza el valor de 0 no debe hacer nada, o sea, la definición base está implícita ya que sólo queremos que haga algo mientras n es mayor a 0. Esto lo hace el paso 2.

Con el paso 2 regresa de recursión y es cuando queremos imprimir el elemento del vector. Observe que el tope de la pila contiene un 1 y justamente el elemento que queremos imprimir es el que está en el índice 0.

Con lo cual, el módulo que imprime los elementos del vector del primero al último será:

```

procedure imprimePrimeroUltimo(vec:vector, n:integer);
begin
    if (n > 0)
    begin
        imprimeUltimoPrimero(vec, n - 1); // Paso 1
2 _____ writeln(vec[n - 1]);        // A esta instrucción regresa el
                                           // control después de que se
                                           // termina una ejecución
                                           // Si n == 0 no hace nada (no
                                           // hay else)
    end;
end;

```

Veamos formalmente cuál es la ejecución de este procedimiento, mostrando la pila de los registros de activación. Normalmente no incluimos en la pila el vector, aunque estrictamente hablando debería estar. Supongamos que el módulo se llama por primera vez desde el *main*, con lo cual la instrucción 1 será la que se encuentre después de esta llamada.

